

Rapport de stage (SPE)

2^{ème} année Cycle Supérieur (2CS)

Option : Systèmes Informatiques (SQ)

Thème :

Enterprise Architecture Knowledge Graphs (EAKGs)

Réalisé par :

–BELGUESMIA Oussama
–KHIARI Anis nassim

Encadré par :

–KHOURI Selma
–SAID LHADJ Lynda

Organisme d'accueil : Laboratoire de la Communication dans les Systèmes Informatiques (LCSI), ESI

Année universitaire : 2026 – 2027

Abstract

Enterprise architecture (EA) models provide an integrated representation of an organization’s business, application, and technology layers, yet traditional modeling tools do not always make this interconnected data easy to analyze. This work presents a methodology for transforming ArchiMate models into Semantic Web Knowledge Graphs (KGs). We introduce a phased translation algorithm that converts ArchiMate Model Exchange Format (XML) into an RDF/OWL ontology, using native RDFS reasoning capabilities. Building on this graph, we deploy a SPARQL-based analytical pipeline directly within the triplestore to support architectural evaluations, including TOGAF Gap Analysis and the detection of structural EA smells. Finally, we evaluate the use of Large Language Models (LLMs) to explore these EA models through natural language. Our experiments contrast static context provisioning (XML, JSON, and Turtle dumps) with interactive, tool-based access via GraphDB. In this setup, the interactive condition achieved a higher mean final score (0.90) than the static representations, suggesting that dynamic access to semantic graphs may be useful for AI-assisted enterprise architecture analysis.

Keywords: Enterprise Architecture; ArchiMate; Knowledge Graphs; RDF/OWL; Large Language Models.

Contents

Abstract	i
List of Figures	iv
List of Tables	v
List of Abbreviations	vii
Introduction	1
1 Translation Methodology: ArchiMate to RDF/OWL	2
1.1 Knowledge Graphs for Enterprise Architecture	2
1.2 Related Work: The EAKG Tool	3
1.2.1 Creation Phase (Knowledge Extraction)	3
1.2.2 Knowledge Graph Enrichment	3
1.2.3 Knowledge Graph Deployment	3
1.3 ArchiMate to RDF: Our Approach	3
1.4 The Base Ontology	4
1.4.1 Relationships in the Ontology	4
1.5 ArchiMate Model Exchange Format	4
1.6 Knowledge Graph Construction Lifecycle	5
1.6.1 Phase 1: Knowledge Graph Creation (Base Graph Construction)	5
1.6.2 Phase 2: Knowledge Graph Enrichment (Semantic Layering)	5
1.7 Core Translation Rules	6
1.7.1 Rule 1: Elements (Vertices)	6
1.7.2 Rule 2: Relations (Edges)	6
1.7.3 Rule 3: Property Definitions and Instances	6
2 Implementation	9

2.1	Architecture of the Translator	10
2.1.1	Namespaces and Lookup Tables	11
2.1.2	Utility Functions	11
2.1.3	Pass 1: Property Definitions (Rule 3 — Definition Half)	12
2.1.4	Pass 2: Elements (Rule 1)	12
2.1.5	Pass 3: Relationships (Rule 2)	12
2.1.6	Pipeline Orchestration	12
2.2	Enrichment: Ontology Loading and Smell Tagging	12
2.2.1	Ontology Loading	12
2.2.2	EA Smell Tagging	13
2.3	Tools and Technology Stack	13
3	SPARQL Querying and Analysis	14
3.1	Simple and Exploratory Queries	14
3.1.1	Dependency Chain Queries	14
3.1.2	Orphan Element Detection	15
3.1.3	Most Connected Elements	16
3.1.4	Layer Listings	17
3.2	Gap Analysis	17
3.2.1	The TOGAF Gap Analysis Technique	17
3.2.2	ArchiMate Representation	18
3.2.3	Knowledge Graph Role in Gap Analysis	19
3.2.4	Example: ArchiSurance Case Study	20
3.2.5	Further Exploration: Ontology-Based EA Evolution	21
3.3	EA Smells Detection Queries	22
3.3.1	Structural and Connectivity Smells	22
3.3.2	Service and Interaction Smells	24
3.3.3	Model Quality Smells	25
3.3.4	Utility Functions	26
4	Natural-Language Access to the Enterprise Architecture Knowledge Graph	27
4.1	Objective and Motivation	27
4.2	Test Setup	28
4.3	Evaluation Methodology	29
4.3.1	Ground Truth	29
4.3.2	Grading Dimensions and Procedure	29
4.4	Results	30
4.5	Synthesis and Limitations	31

4.5.1	Main Findings	31
4.5.2	Limitations	32
Conclusion		33
References		34

List of Figures

1.1	Example: classification of BusinessActor in the ontology hierarchy and its position in the ArchiMate framework	4
1.2	EAKG representation of the input ArchiMate fragment in Listing 1.1	8
2.1	Translator and enrichment pipeline	10
4.1	Experimental setup for the four LLM evaluation conditions	29

List of Tables

3.1	Dependency chain results for “Document Management Back-Up Server”	15
3.2	Eight most connected elements	17
3.3	Relationships from Application Layer to Business Layer	17
4.1	Summary of the LLM evaluation results	31

List of Abbreviations

ABB Architecture Building Block

API Application Programming Interface

DBMS Database Management System

EA Enterprise Architecture

EAKG Enterprise Architecture Knowledge Graph

IRI Internationalized Resource Identifier

JSON JavaScript Object Notation

KG Knowledge Graph

LLM Large Language Model

OWL Web Ontology Language

RDF Resource Description Framework

RDFS RDF Schema

SPARQL SPARQL Protocol and RDF Query Language

TOGAF The Open Group Architecture Framework

TTL Turtle

URI Uniform Resource Identifier

XML Extensible Markup Language

Introduction

Enterprise architecture models offer an integrated and unified representation of the different layers of an organization's activity, thereby facilitating decision-making. These layers structure the vision of business processes, application architecture, technical infrastructure, as well as the interactions between them.

Modeling languages that allow for the mapping of this architecture are widely used, such as ArchiMate, the standard language of the Open Group. However, to fully exploit these models and automate their analysis via complex queries, a transition to Knowledge Graphs (KG) is being adopted [1]. KGs are currently deployed on a large scale, ranging from the massive KGs of web giants (Google, Facebook, or Amazon) to more targeted enterprise KGs.

Tools like the EAKG Toolkit [2] allow ArchiMate models to be transformed into a KG based on the Property Graph model (supported by the Neo4j DBMS). Another KG standard, originating from the Semantic Web, is the RDF graph model (supported by other DBMSs such as GraphDB).

The objective of this project is to:

1. Provide an algorithm allowing the translation of an ArchiMate model according to the RDF standard. Examples of ArchiMate models are provided as input (in XML). The algorithm, inspired by prior work [1], must provide the RDF representation of these models. The user will thus be able to export the resulting RDF KG to a DBMS of their choice (GraphDB in our case).
2. Propose queries to analyze ArchiMate models through the KG. The queries will be formulated in SPARQL, the standard language for querying the RDF model, similar to SQL for querying a relational model.
3. The use of an LLM can be considered to allow querying the obtained KG in natural language—English—(translating the natural language query into the SPARQL language).

Chapter 1

Translation Methodology: ArchiMate to RDF/OWL

This chapter presents the methodology adopted for translating an ArchiMate model to RDF Knowledge Graphs, and designing an OWL ontology to represent ArchiMate standard semantics and allow better usability of the resulting graph. It begins with the motivation for using Knowledge Graphs in enterprise architecture management, reviews the existing EAKG Tool approach, and then details our RDF-based translation rules, the base ontology design, and the phased construction lifecycle.

1.1 Knowledge Graphs for Enterprise Architecture

EA models have a graph structure, but current analysis tools do not take full advantage of this property, which allows differentiation of relations between elements, discovery of paths, clusters, and graph metrics. Moreover, current tools offer a limited set of visualization techniques.

Knowledge Graphs ease advanced reasoning—whether rule-based or machine learning—to provide question-answering capabilities. A model-based construction method combined with graph enrichment can exploit the KG representation and reasoning [1]. Graph visualization can be easily customized, and storing the graph in a graph database enables efficient execution of complex queries. Several graph-based methods have been proposed to analyze EA models, including failure impact analysis with Bayesian Belief Networks and fault trees to analyze the availability of EA components.

1.2 Related Work: The EAKG Tool

A model-based construction method for Enterprise Architecture Knowledge Graphs is proposed in [1]. The approach maps ArchiMate models to a Property Graph representation, following a two-phase lifecycle:

1.2.1 Creation Phase (Knowledge Extraction)

During the creation phase, ArchiMate layers and aspects are mapped to KG properties. Types (classes) are derived from the layer-specific metamodel. KG nodes and edges are derived from the concrete ArchiMate model (element names, connections). Relationships are mapped to edges, and relationship connectors are mapped to properties (AND/OR). Concrete behavioral and structural elements are mapped to nodes, while abstract details are mapped to properties (structure/behavioral element).

1.2.2 Knowledge Graph Enrichment

In the enrichment phase, general graph characteristics are added as properties to nodes, such as centralities, communities (node colors), and node size. Each node is associated with PageRank, betweenness, degree, Louvain, label propagation, and WCC scores, which can be useful for analysis. A color is assigned to each node based on ArchiMate color conventions. Additionally, smells are detected and affected nodes are labeled.

1.2.3 Knowledge Graph Deployment

The resulting graph database (a Neo4j instance) contains the enriched graph with detected smells. The EAKG Toolkit [2] provides an implementation of this pipeline as a plugin for the Archi modeling tool.

Our equivalent mapping from ArchiMate models to RDF Knowledge Graphs is formalized later in Section 1.7.

1.3 ArchiMate to RDF: Our Approach

Following the approach presented in [1], we translate ArchiMate models into RDF Knowledge Graphs. The detailed mapping is defined in Section 1.7; Section 1.4 introduces the ontology, which provides the semantic vocabulary used by those rules.

1.4 The Base Ontology

The ArchiMate framework organizes concepts along two dimensions: *layers* (Strategy, Business, Application, Technology, Physical, and Implementation & Migration) and *aspects* (Passive Structure, Behavior, Active Structure, and Motivation), as defined by the ArchiMate standard [3]. Our ontology maps the ArchiMate metamodel to OWL classes; its class hierarchy mirrors the metamodel’s Layer/Aspect/Concept structure. To illustrate how the ontology classifies architecture elements, Figure 1.1 shows an example for BusinessActor.

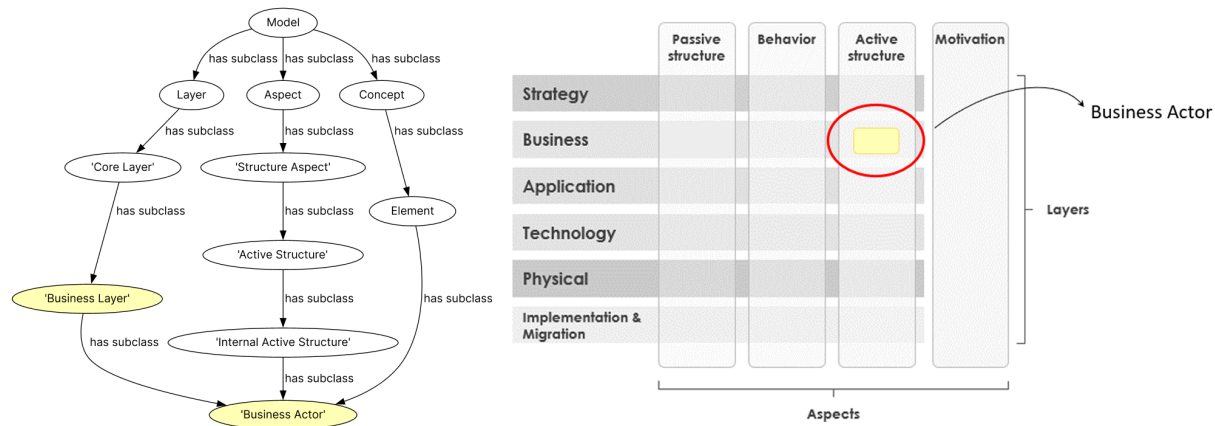


Figure 1.1: Example: classification of BusinessActor in the ontology hierarchy and its position in the ArchiMate framework

1.4.1 Relationships in the Ontology

Relationships are modeled as `owl:ObjectProperty` instances, which connect elements directly. They are organized into four base properties: `structuralRelationship`, `dependencyRelationship`, `dynamicRelationship`, and `otherRelationship`. Relationship connectors (AND, OR) are modeled as a class that is instantiated to create a junction, then assigned the type “and” or “or”; elements participate in the junction through the appropriate relationship.

1.5 ArchiMate Model Exchange Format

The ArchiMate Model Exchange Format stores a model in XML. The top-level model contains separate containers for the information needed by the translator, notably `<propertyDefinitions>`, `<elements>`, and `<relationships>`. Property definitions declare the available property keys and their datatypes. Element records provide identifiers, types, names, documentation, and property values, while relationship records provide identifiers, relationship types, source and target references, and any associated metadata.

The translator reads these containers in a controlled order: property definitions establish

the vocabulary for custom properties, elements establish the resources and their types, and relationships connect those resources. Names and documentation may appear in language-specific `<name>` and `<documentation>` entries, so the corresponding language information is preserved when the XML is converted to RDF literals. The resulting RDF/OWL structure is specified formally in Section 1.7.

1.6 Knowledge Graph Construction Lifecycle

To ensure modularity and extensibility, this translation algorithm formally adopts the phased Model-based Knowledge Graph Construction Process.

1.6.1 Phase 1: Knowledge Graph Creation (Base Graph Construction)

In the Knowledge Graph Creation stage, the relevant information is extracted from the EA model to establish the initial semantic topology of the EAKG. Property definitions are registered first, followed by the creation and typing of element resources and the connection of those resources through relationship predicates. The base graph is grounded using `rdf:type` and `rdfs:subPropertyOf`, defining the ArchiMate entities present and preparing the graph for enrichment. The exact representation of each resource, relationship, and property is given in Section 1.7.

The canonical ArchiMate schema ontology is published at the persistent URL <http://purl.org/eakg/archimate>, which also serves as the Ontology IRI. The PURL redirects to the file's current hosting location, so identifiers stay stable even if the hosting changes. The methodology enforces a strict separation between *static schema* (the published ontology, containing class hierarchies, relationship types, and schema-fixed attribute declarations) and *dynamic output* (the translator, which emits only model-specific instance data such as per-relation predicates and user-defined properties).

1.6.2 Phase 2: Knowledge Graph Enrichment (Semantic Layering)

The Enrichment Phase expands the base graph with the semantic hierarchy and domain-specific analytical tags. It proceeds in two steps.

First, the ArchiMate and EA smell ontologies are loaded into the triplestore directly via their persistent URLs (<http://purl.org/eakg/archimate> and <http://purl.org/eakg/smells>), alongside the translated model. This supplies the `rdfs:subClassOf` and `rdfs:subPropertyOf` hierarchy that the base graph does not contain—for example, connecting `archimate:BusinessActor` through `archimate:ActiveStructureElement` up to `archimate:BusinessLayer`. When an RDFS ruleset is active, the triplestore expands these hierarchies automatically. No offline merge script is needed.

Second, EA smells are detected and tagged via SPARQL. For each of the seven smells defined in the smell ontology, a detection SELECT query identifies the affected resources, and a tagging INSERT query materializes a `smells:hasSmell` triple on each one. The detection queries and their tagging counterparts are presented in Chapter 3.

This differs from the enrichment approach of [1], where computed graph metrics and EA smell labels are materialized by an external pipeline. Our approach performs enrichment directly in the triplestore via SPARQL updates, avoiding the need for a separate enrichment script. Structural graph analytics such as centrality scores remain a potential future extension.

1.7 Core Translation Rules

1.7.1 Rule 1: Elements (Vertices)

In the Creation Phase, an ArchiMate element represents a distinct entity in the EAKG. Specifically, this targets a concrete element of the target ArchiMate model (e.g., a Business Actor or Application Service).

- Each element identifier becomes an `owl:NamedIndividual`; the element is typed with its ArchiMate class and acts as an RDF subject or object. Names and documentation become multilingual `rdfs:label` and `rdfs:comment` triples.

1.7.2 Rule 2: Relations (Edges)

ArchiMate relationships map to edges in the EAKG, acting as the semantic link between elements.

- The relationship identifier becomes an `owl:ObjectProperty` used directly as the source-to-target predicate. Its optional name and documentation are attached to that predicate as multilingual `rdfs:label` and `rdfs:comment` triples. `rdfs:subPropertyOf` connects it to the corresponding ArchiMate relationship type, avoiding RDF-star reification; relationship properties are emitted as direct triples on the relationship predicate.

1.7.3 Rule 3: Property Definitions and Instances

Abstract details of elements and relations are mapped to properties of the KG metamodel [1]. Because each property's name (key) is guaranteed unique within the ArchiMate global property manager [4], this identity is used directly as the property's own predicate:

- A property key becomes an `owl:DatatypeProperty` with an XSD range and an `rdfs:subPropertyOf` link to `archimate:Property`, even when it has no assigned value. Values are emitted as

direct subject–predicate–literal triples, so repeated values remain distinct objects without intermediate nodes.

- **Schema-fixed attributes.** Certain relationship types carry semantic metadata as fixed XML attributes rather than <property> child elements—specifically `accessType` (on `Access`), `isDirected` (on `Association`), and `modifier` (on `Influence`). The schema declaration (type, range, super-property under `archimate:Property`) lives in the static ontology; concrete values are emitted by the translator.

```
1 <propertyDefinitions>
2   <propertyDefinition identifier="propdef-1" type="string">
3     <name xml:lang="en">Vendor</name>
4   </propertyDefinition>
5 </propertyDefinitions>
6
7 <element identifier="id-47045" xsi:type="Node">
8   <name xml:lang="en">Data Acquisition Gateway</name>
9   <properties>
10    <property propertyDefinitionRef="propdef-1">
11      <value>Cisco</value>
12    </property>
13  </properties>
14 </element>
15 <element identifier="id-47052" xsi:type="CommunicationNetwork">
16   <name xml:lang="en">Home & Away LAN</name>
17 </element>
18
19 <relationship identifier="id-99001" xsi:type="Association"
20   source="id-47045" target="id-47052"/>
```

Listing 1.1: Input ArchiMate fragment used in the worked example

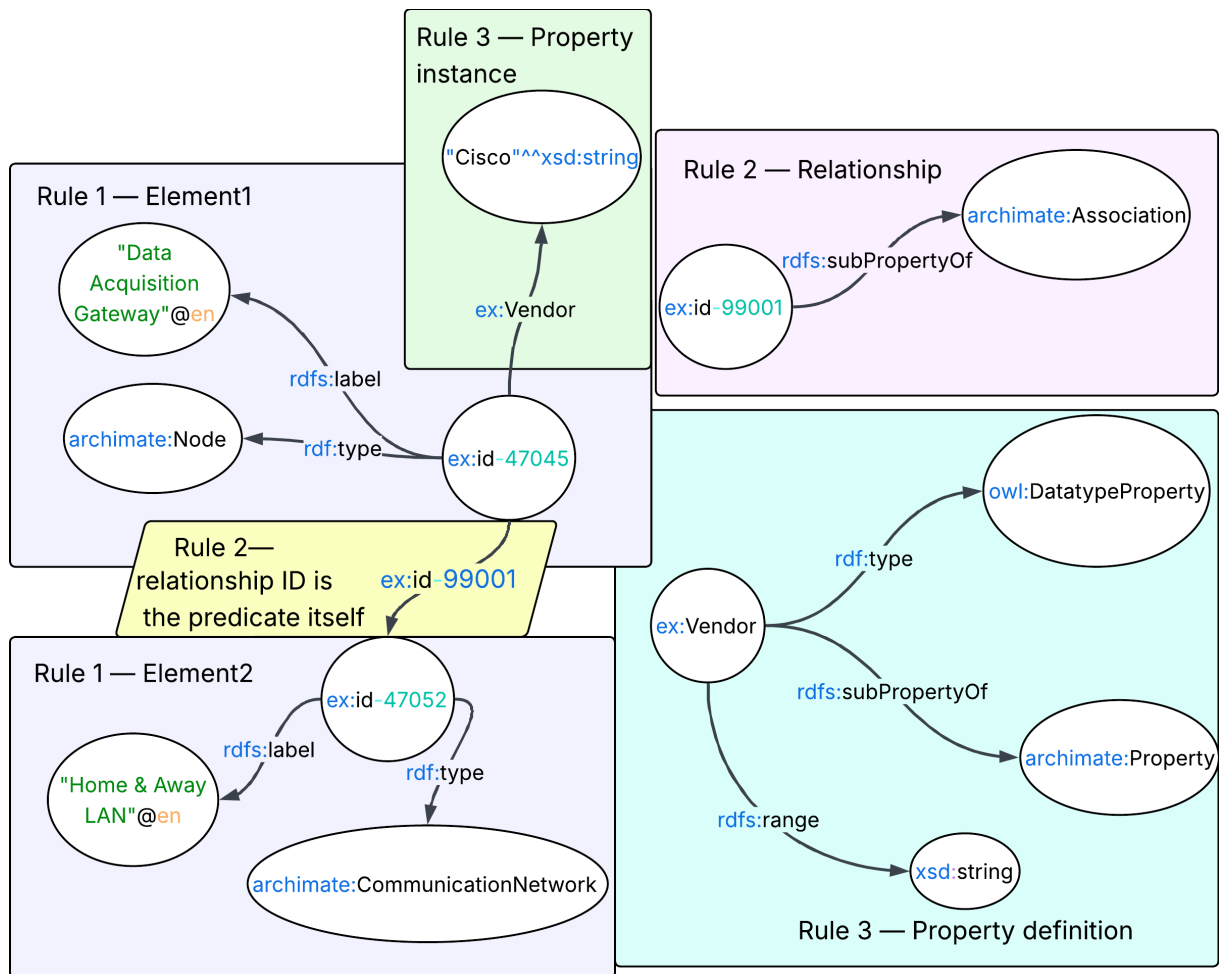


Figure 1.2: EAKG representation of the input ArchiMate fragment in Listing 1.1

Chapter 2

Implementation

This chapter describes the concrete implementation of the translation methodology presented in Chapter 1. The translator (`archimate_to_eakg.py`) converts an ArchiMate Model Exchange File Format XML document into an RDF/OWL knowledge graph. Enrichment is performed directly in the triplestore by loading the ontologies via their persistent URLs and running SPARQL-based smell tagging. The entire stack is built on open standards and open-source tooling.

2.1 Architecture of the Translator

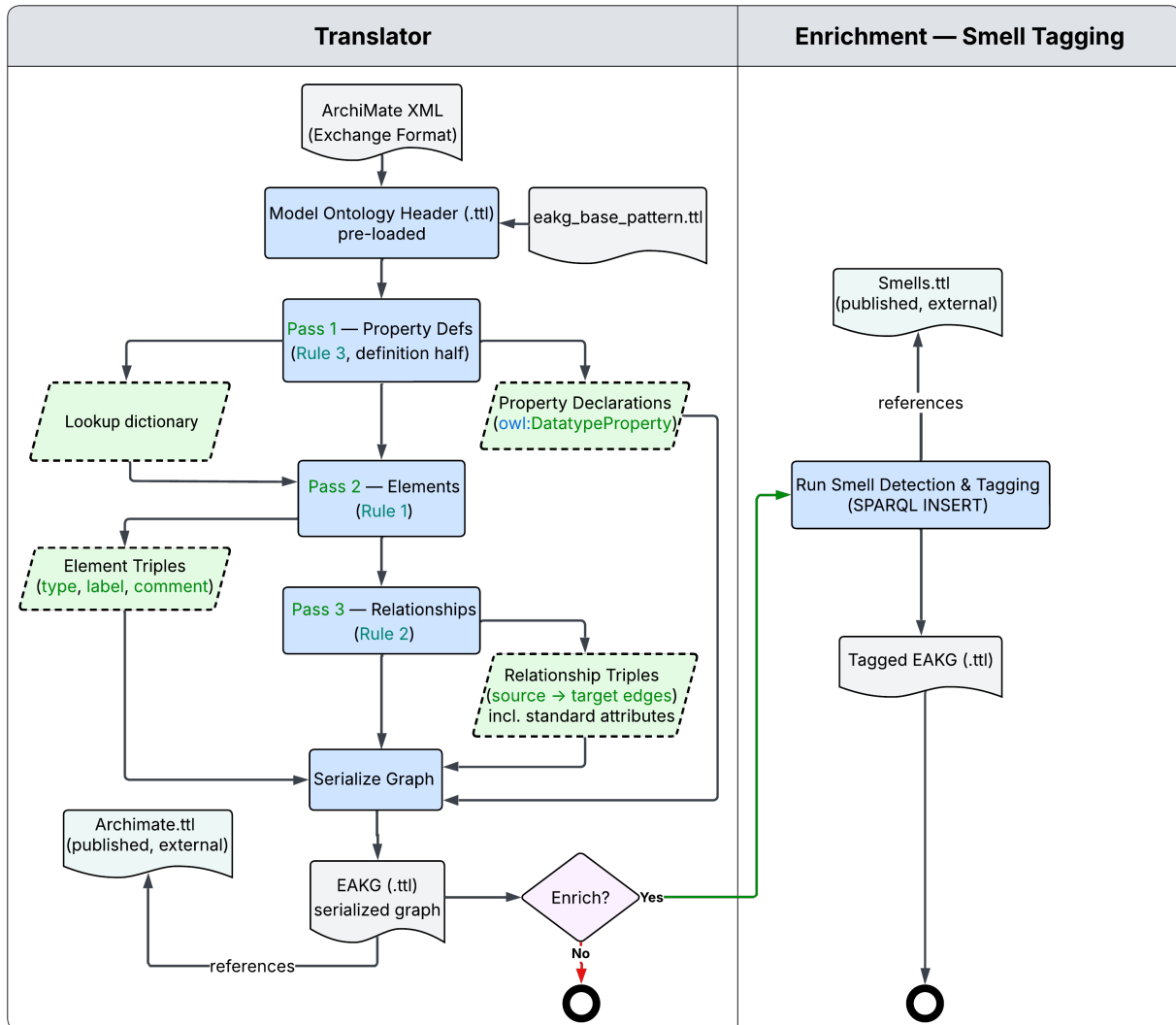


Figure 2.1: Translator and enrichment pipeline

The translator (`archimate_to_eakg.py`) takes an ArchiMate Model Exchange File Format (v3.1) XML file as input and produces an Enterprise Architecture Knowledge Graph serialized as a Turtle (.ttl) file. The program is invoked from the command line:

```
python archimate_to_eakg.py <input.xml> [output.ttl]
```

If no output path is given, the result is written to `eakg_output.ttl` in the same directory as the input file.

The translation pipeline is organized into three sequential passes over the XML document, each implementing one of the core translation rules defined in Chapter 1. Before those passes execute, the program initializes an RDF graph pre-loaded with the base ontology pattern

(`eakg_base_pattern.ttl`), which provides the ArchiMate class hierarchy and base property definitions.

2.1.1 Namespaces and Lookup Tables

The program begins by declaring the RDF namespaces used throughout the graph. Three namespaces are central to the output:

- `ex:` (`http://www.example.org/eakg#`) — used for all individual elements, relationships, and property predicates that originate from the concrete ArchiMate model.
- `archimate:` (`http://purl.org/eakg/archimate#`) — the ArchiMate metamodel vocabulary, published at a persistent URL. Classes such as `archimate:BusinessActor`, relationship types such as `archimate:Association`, and schema-fixed attributes such as `archimate:accessType` are defined under this namespace.
- `smells:` (`http://purl.org/eakg/smells#`) — the EA smell ontology, also published at a persistent URL. Defines the `smells:hasSmell` predicate and seven smell individuals used during the enrichment phase.

A lookup table maps ArchiMate primitive type strings to their corresponding XSD datatype URIs. This table is used during Pass 1 to assign the correct `rdfs:range` to each property definition:

```
"string"  -> xsd:string
"boolean" -> xsd:boolean
"integer"  -> xsd:integer
"float"    -> xsd:float
"date"     -> xsd:date
"time"     -> xsd:time
"number"   -> xsd:decimal
```

2.1.2 Utility Functions

Several helper functions are shared across passes:

- `id_to_iri` and `property_name_to_iri` create stable, sanitized IRIs; `resolve_archimate_class` maps XML types to ontology classes; and `emit_lang_literals` preserves language-tagged names and documentation. Detailed behavior is summarized in Annex B.

2.1.3 Pass 1: Property Definitions (Rule 3 — Definition Half)

The first pass registers property predicates and their XSD ranges, returning the lookup dictionary needed by the element and relationship passes.

2.1.4 Pass 2: Elements (Rule 1)

The second pass converts each XML element into a typed named individual, emits multilingual labels and comments, and writes its property values as direct triples using the Pass 1 dictionary.

2.1.5 Pass 3: Relationships (Rule 2)

The third pass turns each relationship identifier into an object-property predicate, links it to its ArchiMate relationship class, and emits the source-to-target triple plus any metadata and property values.

2.1.6 Pipeline Orchestration

The main entry point ties the pipeline together. It begins by loading the base ontology pattern from the Turtle file `eakg_base_pattern.ttl`, which pre-populates the graph with the generic construction pattern and base property declaration (`archimate:Property`). The ArchiMate class hierarchy is supplied separately by loading the ontology into the triplestore via its PURL (Section 2.2). The entry point then parses the input XML, executes the three passes in order, and serializes the resulting graph to the output Turtle file with all namespace prefixes bound for clean, human-readable output.

The sequential ordering of the passes is deliberate: Pass 1 must complete before Passes 2 and 3 because both element and relationship processing depend on the property definition dictionary built during Pass 1.

2.2 Enrichment: Ontology Loading and Smell Tagging

Enrichment is performed directly in the triplestore rather than through offline merge scripts. It proceeds in two steps.

2.2.1 Ontology Loading

The ArchiMate ontology and the EA smell ontology are loaded into the triplestore via their persistent URLs (<http://purl.org/eakg/archimate> and <http://purl.org/eakg/smells>), alongside the translated model. This supplies the `rdfs:subClassOf` and `rdfs:subPropertyOf` hierarchy required by the reasoning-dependent queries in Chapter 3. No offline merge script is

needed.

2.2.2 EA Smell Tagging

Once the ontologies and model are loaded and RDFS reasoning is active, EA smells are detected and tagged using SPARQL queries. For each smell, the process follows two steps:

1. A **detection** SELECT query identifies the affected elements or relations.
2. A **tagging** INSERT query materializes a `smells:hasSmell` triple on each affected resource, using the same detection logic as its WHERE clause.

The detection queries are unchanged from the analytical queries presented in Chapter 3. The tagging queries are presented alongside them. Once tagged, affected resources can be retrieved with a single generic query by specifying the smell URI, and tags can be selectively reset without affecting other enrichment results.

2.3 Tools and Technology Stack

The implementation relies on the following tools and technologies:

- **Python 3** as the programming language for both the translator and the enrichment module.
- **rdflib** (Python library) for constructing, manipulating, and serializing RDF graphs [5]. It provides native support for Turtle, N-Triples, and other RDF serialization formats, as well as namespace management and literal typing.
- **xml.etree.ElementTree** (Python standard library) for parsing the ArchiMate Model Exchange File Format XML documents.
- **GraphDB** (Ontotext) as the target triplestore for deploying the resulting EAKG [6]. GraphDB supports SPARQL 1.1 querying and RDFS reasoning, which enables the sub-property inference chains relied upon by our translation rules.
- **SPARQL 1.1** as the query language for analyzing the deployed knowledge graph.
- **RDFS reasoning** for inferring implicit relationships through the `rdfs:subPropertyOf` and `rdfs:subClassOf` hierarchies defined in the base ontology. This allows queries to match, for example, all `archimate:DependencyRelationship` instances without enumerating every concrete sub-property.

Chapter 3

SPARQL Querying and Analysis

This chapter demonstrates how the EAKG, once deployed in a SPARQL-enabled triplestore such as GraphDB, can be queried and analyzed. We begin with simple exploratory queries, then present a gap analysis technique that leverages the knowledge graph to automate parts of the TOGAF ADM process, and finally describe a set of EA smell detection queries.

3.1 Simple and Exploratory Queries

3.1.1 Dependency Chain Queries

Dependency chain queries discover dependencies between elements. These dependencies can be direct or follow a transitive chain using the SPARQL property path operator (+). For example, the following query finds all elements that a given component (here, “Document Management Back-Up Server”) depends on, traversing both `archimate:DependencyRelationship` and `archimate:StructuralRelationship` edges:

```
1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
3 PREFIX archimate: <http://purl.org/eakg/archimate#>
4 PREFIX smells: <http://purl.org/eakg/smells#>
5
6 SELECT DISTINCT ?component ?service ?serviceName
7 WHERE {
8     ?component rdfs:label "Document_Management_Back-Up_Server"@en .
9     ?service (archimate:DependencyRelationship
10              |archimate:StructuralRelationship)+ ?component .
11     ?service rdf:type archimate:Element ;
```

```

12         rdfs:label ?serviceName .
13     }

```

Listing 3.1: Dependency chain: elements that a given component depends on

All subsequent queries in this chapter and in the Annex assume the same prefixes unless otherwise noted.

Dependency chains can also be traversed in the opposite direction. For example, the query below lists what depends on a given component (here, “PaaS Services”):

(full query: see Annex, Listing A.1)

Table 3.1: Dependency chain results for “Document Management Back-Up Server”

component	service	service name
ex:id-47094	ex:id-47052	Home & Away LAN
ex:id-47094	ex:id-47506	Homeowner’s & Travel Back Office
ex:id-47094	ex:id-47045	Data Acquisition Gateway
ex:id-47094	ex:id-47088	FO General-Purpose Server
ex:id-47094	ex:id-47090	FO Web Hosting Server
ex:id-47094	ex:id-47508	Home & Away Headquarters
ex:id-47094	ex:id-47100	PaaS Services
ex:id-47094	ex:id-47393	Project: Data Warehousing and BI
ex:id-47094	ex:id-47501	Front Office

3.1.2 Orphan Element Detection

A useful model quality check is to find elements that have no relationship with any other element— orphan nodes that may have been forgotten during modeling:

```

1 SELECT ?element ?elementName
2 WHERE {
3     ?element rdfs:label ?elementName ;
4         a archimate:Element .
5     FILTER NOT EXISTS {
6         ?element archimate:Relationship ?target .
7         ?target a archimate:Element .
8     }
9     FILTER NOT EXISTS {
10        ?source archimate:Relationship ?element .

```

```

11         ?source a archimate:Element .
12     }
13 }

```

Listing 3.2: Detect orphan elements (no incoming or outgoing relationships)

The query returns 15 orphan elements in the ArchiSurance model, including Product Information, Insurance, Insurance Policy, Commission, Bank, Business Intelligence Team, and several customer-channel elements.

3.1.3 Most Connected Elements

Sorting elements by their total number of relationships (incoming and outgoing) helps identify the most central components of the architecture:

```

1 SELECT ?element ?elementName
2       ((COALESCE(?in,0) + COALESCE(?out,0)) AS ?relations)
3 WHERE {
4     ?element rdfs:label ?elementName .
5     ?element a archimate:Element .
6     OPTIONAL {
7         SELECT ?element (COUNT(DISTINCT ?s) AS ?in)
8         WHERE {
9             ?s ?rel ?element .
10            ?rel rdfs:subPropertyOf* archimate:Relationship .
11        }
12        GROUP BY ?element
13    }
14    OPTIONAL {
15        SELECT ?element (COUNT(DISTINCT ?t) AS ?out)
16        WHERE {
17            ?element ?rel ?t .
18            ?rel rdfs:subPropertyOf* archimate:Relationship .
19        }
20        GROUP BY ?element
21    }
22 }
23 ORDER BY DESC(?relations)

```

Listing 3.3: Rank elements by total number of relationships

Table 3.2: Eight most connected elements

element	element name	relations
ex:id-46339	Target: CRM, Back Office and Data Warehouse Operational	36
ex:id-46304	ArchiSurance Back Office Suite	23
ex:id-47396	Baseline	23
ex:id-46637	Finance	18
ex:id-46860	Document Management System	16
ex:id-46330	Manage Policies and Claims	15
ex:id-46918	Home & Away Policy Administration	15
ex:id-46369	Serve Customers	14

3.1.4 Layer Listings

The following query lists all elements together with their ArchiMate layer classification, exploiting the ontology's class hierarchy:

(full query: see Annex, Listing A.2)

Elements can also be listed by the relationships between layers. The cross-layer count below reports the four Application-to-Business type pairs returned by the query.

(full query: see Annex, Listing A.3)

Table 3.3: Relationships from Application Layer to Business Layer

source type	target type	count
archimate:ApplicationComponent	archimate:BusinessFunction	22
archimate:ApplicationService	archimate:BusinessProcess	16
archimate:DataObject	archimate:BusinessObject	3
archimate:ApplicationProcess	archimate:BusinessProcess	1

3.2 Gap Analysis

3.2.1 The TOGAF Gap Analysis Technique

According to the *TOGAF*® Standard—ADM Technique, Gap Analysis is a technique used to validate an architecture that is being developed [7]. The basic premise is to highlight a shortfall between the Baseline Architecture and the Target Architecture; that is, items that have been deliberately omitted, accidentally left out, or not yet defined.

In other words, Gap Analysis is the process of comparing the current architecture baseline with the new architecture we want to achieve in the future (target) to identify what needs to be added, removed, or modified. A key step in validating the architecture is to identify what has been forgotten.

Suggested Method by the Standard

The standard suggests the following matrix-based method:

- Draw a matrix where the Architecture Building Blocks (ABBs) from the baseline architecture are listed per rows, and ABBs present in the target architecture are listed per columns.
- Add a last row labeled “New” and a last column labeled “Eliminated.”
- ABBs available in both architectures are marked with “Included” at the intersecting cell.
- ABBs that appear only in the baseline architecture should be reviewed to determine if they were correctly removed or forgotten. If correctly eliminated, mark them in the appropriate “Eliminated” cell; otherwise they must be readdressed in the next iteration.
- ABBs that appear only in the target architecture are marked in the intersecting cell with the “New” row as a gap.

When the matrix is completed, “Eliminated” and “New” marks represent gaps that should be explained—either as correctly eliminated items or as items that need to be addressed by reinstating or developing/procuring the new building block.

3.2.2 ArchiMate Representation

In ArchiMate, the “Implementation and Migration Layer” includes elements that model the evolution of enterprise architecture through programs and projects [3]. The relevant element types are:

1. **Work Package:** a series of actions identified and designated to achieve specific results within specified time and resource constraints. It has a start and end, and produces goals, outcomes, or deliverables.
2. **Deliverable:** represents a result of a Work Package—reports, papers, services, software, physical products, etc.
3. **Plateau:** represents a relatively stable state of the architecture that exists during a limited period of time. Plateaus show the enterprise at incremental states reflecting periods of

transition between the Baseline and Target Architectures, and allow grouping work packages into portfolios and programs.

4. **Gap:** represents a statement of difference between two plateaus. It is associated with two plateaus and represents the difference between them.

3.2.3 Knowledge Graph Role in Gap Analysis

By transforming the ArchiMate model into a knowledge graph, we can automate the creation of the matrix in the Gap Analysis process. By querying the Knowledge Graph we can extract the list of “Kept,” “Eliminated,” and “New” ABBs. After evaluation by the architect, those elements are marked by linking them to the “Gap” node with the appropriate property, to keep track of taken decisions in the knowledge graph.

Queries to List Gap Elements

Three queries extract the gap classification directly from the knowledge graph:

1. **ABBs present in both architectures (Kept).** This query returns the elements aggregated by both the baseline and target plateaus. (*full query: see Annex, Listing A.4*)
2. **ABBs present only in the baseline (candidates for elimination).** This query identifies elements that belong to the baseline plateau but are absent from the target plateau. (*full query: see Annex, Listing A.5*)
3. **ABBs present only in the target (new gaps).** This query identifies elements introduced in the target plateau that do not exist in the baseline plateau. (*full query: see Annex, Listing A.6*)

Marking Elements After Review

We use project-specific RDF properties to link the “Gap” node with each element that presents a gap, which allows us to mark elements with the appropriate decision after review:

- `ex:gapNew` for new ABBs to be produced/procured.
- `ex:gapInclude` for ABBs to be kept, which has a sub-property `ex:gapImproved` for kept ABBs that require improvement.
- `ex:gapEliminated` for ABBs correctly removed from the architecture.

3.2.4 Example: ArchiSurance Case Study

Using the ArchiSurance case study [8], we can model the evolution with a “Gap” between two plateaus:

- The **baseline plateau** aggregates: Web Portal, Call Center Application, Legal Expense CRM System, General CRM System, Home & Away Financial Application, Home & Away Policy Administration, Auto Insurance Application, Legal Expense Back-Office System, Document Management System.
- The **target plateau** aggregates: Web Portal, Call Center Application, General CRM System, Document Management System, ArchiSurance Back-Office Suite, Data Warehousing Solution, MyArchiSurance Mobile App.

In the knowledge graph, each plateau is represented with a node that has an “aggregate” property linking it to all its included elements, and a “Gap” node is associated with both plateau nodes. Using the previous queries, we can list the gap classification without the need to draw the matrix:

- **Kept:** Web Portal, Call Center Application, General CRM System, Document Management System.
- **Eliminated:** Legal Expense CRM System, Home & Away Financial Application, Home & Away Policy Administration, Auto Insurance Application, Legal Expense Back-Office System.
- **New:** ArchiSurance Back-Office Suite, Data Warehousing Solution, MyArchiSurance Mobile App.

The following combined query classifies elements from the gap analysis according to the TOGAF gap analysis matrix, using concrete identifiers from the ArchiSurance model:

```
1 # Newly introduced prefix for this query.
2 PREFIX ex: <http://www.example.org/eakg#>
3
4 SELECT ?ABB ?label ?state
5 WHERE {
6     BIND(ex:id-46339 AS ?target)
7     BIND(ex:id-47396 AS ?baseline)
8     {
9         ?baseline archimate:Aggregation ?ABB .
10        ?target archimate:Aggregation ?ABB .
```

```

11         ?ABB rdfs:label ?label .
12         BIND("Kept" AS ?state)
13     }
14     UNION
15     {
16         ?baseline archimate:Aggregation ?ABB .
17         FILTER NOT EXISTS {
18             ?target archimate:Aggregation ?ABB .
19         }
20         ?ABB rdfs:label ?label .
21         BIND("Eliminated" AS ?state)
22     }
23     UNION
24     {
25         ?target archimate:Aggregation ?ABB .
26         FILTER NOT EXISTS {
27             ?baseline archimate:Aggregation ?ABB .
28         }
29         ?ABB rdfs:label ?label .
30         BIND("New" AS ?state)
31     }
32 }

```

Listing 3.4: Classify elements as Kept, Eliminated, or New in one query

After reviewing each element, we mark it with the appropriate property by inserting triples with the “Gap” node. The following SPARQL Update query shows how decisions are recorded:

(full query: see Annex, Listing A.7)

3.2.5 Further Exploration: Ontology-Based EA Evolution

A small ontology to model enterprise evolution is proposed in [9], taking advantage of computer agents to aid communication and analysis of EA model evolution. The key concepts of this ontology are:

- An **IT Project** acts as an enabler of EA evolution by applying a set of **Transformations**.
- A Transformation can be decomposed into smaller **Changes**, each altering the life-cycle of an EA Description (element or relationship).

- To keep track of motivation and interested parties, the ontology introduces **Stakeholder** (who has one or more drivers) and **Driver** (an internal or external motivation/condition that makes the change necessary).
- This model keeps information about how the EA was changed, when those changes happened, who involved those changes, and why.

The authors proposed a set of Description Logic (DL) queries to answer questions that help decision-making and facilitate change analysis. In our case, DL queries can be replaced with equivalent SPARQL queries into the knowledge graph.

For example, to find what transformations are applied by an IT project named “Back-Office System Integration,” the proposed DL query:

```

1 Transformation and
2   applied_by some
3   (IT_Project and
4     hasName value "Back-Office_System_Integration")

```

can be expressed as the following SPARQL equivalent:

(full query: see Annex, Listing A.8)

3.3 EA Smells Detection Queries

Enterprise Architecture Smells are recurring patterns in an EA model that indicate potential design problems [10]. The following subsections present SPARQL queries that detect and tag several categories of smells, leveraging the RDFS reasoning capabilities of the triplestore. For each smell, a detection SELECT identifies the affected resources and a tagging INSERT materializes a `smells:hasSmell` triple on each one. These queries were tested on the demonstration model from [10], which was designed to exhibit each smell.

3.3.1 Structural and Connectivity Smells

Cyclic Dependency

The Concept: Elements should not be stuck in an endless loop of depending on one another. This behavior is illustrated in the demonstration model of [10].

The Detection Logic: We target the `archimate:DependencyRelationship` hierarchy and scan the graph globally to match unbroken chains of these dependency edges. If traversing that chain eventually leads right back to the exact same starting node, a cycle is detected.

(full query: see Annex, Listing A.9)

Each element in the cycle is tagged with `smells:CyclicDependency`. *(tagging query: see Annex, Listing A.14)*

Shared Persistency

The Concept: Higher-level business components should not bypass the application layer to talk directly to the same backend database. This behavior is illustrated in the demonstration model of [10], where it is treated together with Strict Layers Violation rather than as a separately defined smell.

The Detection Logic: We isolate `archimate:PassiveStructureElement` nodes, which represent data. We then look for instances where two or more distinct elements from the `archimate:BusinessLayer` have an `archimate:Access` edge pointing directly to that exact same data node.

(full query: see Annex, Listing A.10)

The shared data node is tagged with `smells:SharedPersistency`. *(tagging query: see Annex, Listing A.15)*

Strict Layers Violation

The Concept: The architecture should follow a strict top-down hierarchy (Business → Application → Technology). Skipping an intermediate layer by establishing a direct link between the Business layer and the Technology layer breaks this core design principle. As a best practice, no direct architectural relationships of any kind or direction should bridge these layers directly without traversing the application layer.

The Detection Logic: We search for any direct relationship where one connected node belongs to the `archimate:BusinessLayer` and the other belongs to the `archimate:TechnologyLayer`. By checking both directions, we confirm that the `archimate:ApplicationLayer` has been entirely bypassed, regardless of how the relationship was originally oriented in the model.

```
1 SELECT DISTINCT ?nodeA ?nodeB ?relation ?direction
2 WHERE {
3     {
4         ?nodeA a archimate:BusinessLayer .
5         ?nodeB a archimate:TechnologyLayer .
6         ?nodeA ?relation ?nodeB .
7         BIND("Business_to_Tech" AS ?direction)
8     }
9     UNION
```

```

10 {
11     ?nodeA a archimate:TechnologyLayer .
12     ?nodeB a archimate:BusinessLayer .
13     ?nodeA ?relation ?nodeB .
14     BIND("Tech_to_Business" AS ?direction)
15 }
16 ?relation rdfs:subPropertyOf* archimate:Relationship .
17 FILTER(STRSTARTS(STR(?relation),
18     "http://www.example.org/eakg#id"))
19 }

```

Listing 3.5: Detect strict layers violation

The offending relationship edge is tagged with `smells:StrictLayersViolation`. (*tagging query: see Annex, Listing A.16*)

3.3.2 Service and Interaction Smells

Chatty Service

The Concept: A service should not be overly communicative, firing off a massive number of granular requests to a vast array of other services. [10] state that a service related to a high number of other services may indicate this smell.

The Detection Logic: We target service nodes (`archimate:ApplicationService`, `archimate:BusinessService`) and count the total number of unique elements they connect to. If that count exceeds a defined acceptable threshold, the service is flagged.

```

1 SELECT ?service ?label
2     (COUNT(DISTINCT ?connectedElement) AS ?connectionCount)
3 WHERE {
4     ?service a ?serviceType .
5     FILTER (?serviceType IN (archimate:ApplicationService,
6                             archimate:BusinessService))
7     ?service rdfs:label ?label .
8     {
9         ?service ?relation ?connectedElement .
10    }
11    UNION
12    {
13        ?connectedElement ?relation ?service .
14    }

```

```

15     ?relation rdfs:subPropertyOf* archimate:Relationship .
16     FILTER(STRSTARTS(STR(?relation),
17         "http://www.example.org/eakg#id"))
18 }
19 GROUP BY ?service ?label
20 HAVING (COUNT(DISTINCT ?connectedElement) >= 10)

```

Listing 3.6: Detect chatty services (threshold: 10 connections)

The overly communicative service node is tagged with `smells:ChattyService`. (*tagging query: see Annex, Listing A.17*)

Data Service

The Concept: A service should execute logic. If it only touches data objects and nothing else, it is just a weak data-access point. This smell is described in [10] as a service that only references data objects without relating to other components.

The Detection Logic: We isolate service nodes and evaluate their *outgoing* relationships (where the service acts as the source). We identify services that have outgoing edges pointing to `archimate:PassiveStructureElement` nodes, but apply a strict filter to ensure they have *zero* outgoing edges pointing to any `archimate:ActiveStructureElement` or `archimate:BehaviorElement`. Incoming edges (other components calling the service) are expected and are ignored by this filter.

(*full query: see Annex, Listing A.11*)

The data-only service node is tagged with `smells:DataService`. (*tagging query: see Annex, Listing A.18*)

3.3.3 Model Quality Smells

Duplication

The Concept: The model should not contain multiple distinct elements that share the exact same name. Sharing a name alone is treated as a smell, regardless of the elements' specific types. A similar case is illustrated in the demonstration model of [10], based on name similarity rather than exact matching.

The Detection Logic: We compare the `rdfs:label` (the name) across the graph. If we find two or more distinct nodes (different IRIs) sharing the exact same label, they are flagged as duplicates.

(*full query: see Annex, Listing A.12*)

Each element sharing its label with at least one other element is tagged with `smells: Duplication`.
(tagging query: see Annex, Listing A.19)

Unnecessary Documentation

The Concept: Model documentation should be concise. Huge walls of text clutter the repository. This smell is illustrated in the demonstration model of [10], but no general threshold is stated.

The Detection Logic: We extract the `rdfs:comment` string literal for every element in the graph, calculate the character length of that string, and flag comments that exceed a specific character limit (e.g., 400 characters).

(full query: see Annex, Listing A.13)

Any model resource exceeding the character limit is tagged with `smells: UnnecessaryDocumentation`.
(tagging query: see Annex, Listing A.20)

3.3.4 Utility Functions

The following query lists the elements tagged with a selected smell; replace `smells: CyclicDependency` with any smell URI as needed:

```
1 SELECT ?element ?label WHERE {  
2     ?element smells:hasSmell smells:CyclicDependency .  
3     OPTIONAL { ?element rdfs:label ?label . }  
4 }
```

Listing 3.7: View tagged elements for a specific smell

A corresponding query for resetting a selected smell tag is provided in Annex, Listing A.21.

Chapter 4

Natural-Language Access to the Enterprise Architecture Knowledge Graph

4.1 Objective and Motivation

The third objective of this work is to evaluate whether Enterprise Architecture models can be effectively explored and queried through natural-language interaction with a Large Language Model (LLM), and to what extent the representation format of the model—raw, translated, or made queryable—affects the quality of the answers produced.

While the previous objectives focused on defining a translation methodology from ArchiMate models to a Semantic Web ontology (RDF/OWL) and enriching the resulting Enterprise Architecture Knowledge Graph (EAKG) with structural and domain-specific knowledge such as EA smells, this objective addresses a practical question: does this translation effort improve an LLM’s ability to reason about an EA model compared with simply providing the model in its native exchange format? Furthermore, does giving the LLM active access to the graph, rather than a static dump of it, change its performance?

To answer this, four experimental conditions were designed by varying two independent dimensions:

- **Representation:** the raw ArchiMate Exchange Format model versus the translated EAKG ontology.
- **Access mode:** the full model provided statically as context versus the model made available as a live, queryable resource through a tool.

This comparison isolates the respective contributions of the translation methodology and

interactive graph access rather than conflating the two.

4.2 Test Setup

The four conditions were as follows:

1. **Raw XML (static context).** The ArchiMate Exchange Format model, exported directly from the modeling tool, was provided in full as context. No translation or ontology mapping was applied.
2. **Raw JSON (static context).** The same exchange-format model, converted to an equivalent JSON representation, was provided in full. This condition isolates XML versus JSON serialization while keeping the underlying information identical.
3. **Translated EAKG ontology (static context).** The model was passed through the translation pipeline defined in this work, producing the EAKG in RDF/OWL. The resulting Turtle graph was provided as static context together with a general explanation of its namespaces, element and relationship mapping, property mapping, and layer and category resolution. No ArchiSurance-specific information was included in that explanation.
4. **GraphDB live access (interactive).** The translated EAKG was loaded into GraphDB. The LLM received a SPARQL-query tool and the same schema explanation as in Test 3, and had to retrieve the information needed for each answer.

Figure 4.1 summarizes the four experimental conditions and the distinction between static context and interactive graph access.

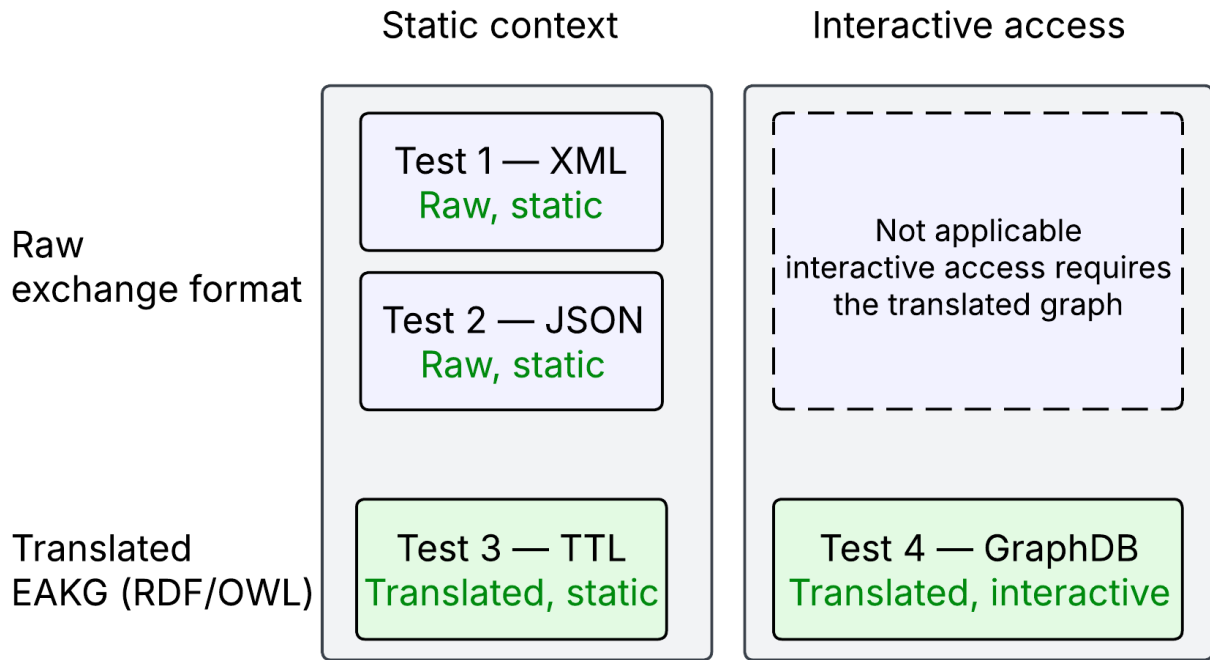


Figure 4.1: Experimental setup for the four LLM evaluation conditions

4.3 Evaluation Methodology

Answers produced under each condition were scored using a dedicated LLM-based grading procedure applied uniformly across all four tests.

4.3.1 Ground Truth

The original ArchiSurance model in its native exchange format served as the sole authority for element identity, type, layer, and relationship existence, type, and direction. The accompanying Open Group ArchiSurance documentation was used only to resolve ambiguity and clarify intended semantics, never as a source of exact identifiers.

Although the grader was also an LLM, its role differed from that of the models under evaluation. The generation LLM in each test performed open-ended interpretation of a model it had to understand and reason about. The grading LLM was given the complete ground-truth model and performed a comparison task: it checked whether the elements, relationships, and directions cited in an answer matched what the ground truth explicitly contained. It was therefore verifying specific claims rather than independently reconstructing architectural knowledge.

4.3.2 Grading Dimensions and Procedure

Each answer was scored independently along three axes:

- **Correctness** (0.00–1.00): how completely and accurately the answer matched what the ground truth supported, including penalties for missing elements, wrong relationship directions, and vague answers.
- **Grounding** (0.00–1.00): whether every substantive claim was traceable to a specific element or relationship identifier in the model.
- **Hallucination** (true/false): whether the answer contained an invented element, relationship, identifier, or type not present in the ground truth.

The final score was derived as a weighted combination of Correctness and Grounding using weights of 0.7 and 0.3. When hallucination was detected, the result was capped at 0.25. A correct refusal was scored 1.0 directly because it contained no specific claim to ground; if it also involved a hallucination, the same 0.25 cap applied.

For each question, the grader first determined the correct answer from the ground-truth model, then compared the generated answer with that reference. Special handling covered correct refusals, partial or missing identifiers, directional errors, and, for the interactive condition, answers that described a query strategy without reporting the instance-level results retrieved from the graph. Failed runs caused by tool or API errors were excluded from scoring and logged separately rather than penalized as incorrect answers.

The grading prompts for Tests 1–3 and Test 4 shared the same rubric, dimensions, score formula, and core edge-case rules. The Test 4 version additionally specified how to handle tool or API failures and query-strategy descriptions, which do not arise in the static-context conditions.

While the scoring was automated, targeted manual validation was performed on edge cases and ambiguous responses to ensure the integrity of the final scores.

4.4 Results

Each test was run twice on the same 26 questions. Answers were scored from 0 to 1, with correctness and grounding graded separately and hallucination flags recorded for each answer. Table 4.1 summarizes the results; the complete question-level grid is provided in Annex C.

Table 4.1: Summary of the LLM evaluation results

Test	Run 1	Run 2	Mean final score	Mean correctness	Mean grounding	Hallucinations
1 XML (static)	0.70	0.90	0.80	0.83	0.86	7/52 (13%)
2 JSON (static)	0.74	0.88	0.81	0.81	0.91	4/52 (8%)
3 TTL (static)	0.58	0.75	0.67	0.72	0.82	12/52 (23%)
4 GraphDB	0.89	0.91	0.90	0.95	0.73	1/51 (2%)

Test 4 has 51 scored answers because one answer in Run 1 failed with a 503 error and was excluded. GraphDB access obtained the highest score and was the most stable across runs. The static Turtle dump obtained the lowest score, while XML and JSON were close to each other between the two extremes.

4.5 Synthesis and Limitations

4.5.1 Main Findings

Access mode mattered more than format. Tests 3 and 4 used the same graph and schema explanation, yet GraphDB scored about 0.23 higher (0.67 versus 0.90) and produced no zero-score answers. The ontology was more useful when queried than when read as a static text dump.

XML and JSON performed similarly. Their mean scores were close (0.80 versus 0.81). JSON had better grounding (0.91 versus 0.86) and fewer hallucination flags (4 versus 7 of 52), whereas XML had slightly higher correctness (0.83 versus 0.81). With only two runs per condition, these differences indicate at most a mild advantage for JSON.

Static conditions were less stable. Run 2 exceeded Run 1 by 0.20 for XML, 0.14 for JSON, and 0.17 for TTL, whereas GraphDB changed by only 0.02. This pattern is suggestive but not conclusive.

Failure modes differed by condition. XML and JSON produced wrong relationship targets, wrong element types, and refusals for information present in the model. TTL mainly produced relation errors, including reversed directions and invented paths, as well as corrupted identifiers. GraphDB produced almost no direction or identifier errors; its only hallucination was a relation between real elements that was not present in the graph.

Correctness and grounding should be read separately. GraphDB had the highest correctness (0.95) but the lowest grounding (0.73). Several answers described the query strategy rather than tracing the instance identifiers behind the result, which reduced their grounding

scores even when the final answer was correct.

4.5.2 Limitations

Question 1 was graded inconsistently across files. When the exact answer was absent, some answers refused and offered a close alternative, while the rubric did not clearly specify how to treat that behavior. The question was retained in the averages, and excluding it did not change the ranking.

The potential bias introduced by the LLM-based grader, as well as the scope of the manual verification, is discussed further in the Conclusion.

Finally, two runs per condition are insufficient for significance testing. Differences smaller than approximately 0.05 should therefore not be interpreted as reliable performance differences.

Conclusion

This project connects standard Enterprise Architecture modeling with Semantic Web-based analysis. By formalizing a translation pipeline from ArchiMate XML to an RDF/OWL Knowledge Graph, we established a semantic basis for architectural analysis. The implementation shows how knowledge enrichment—such as automating TOGAF Gap Analysis and identifying EA smells—can be executed directly within the triplestore via SPARQL updates to support the tracking of architectural evolution.

Furthermore, our LLM evaluation yields preliminary insights into natural-language model exploration. In our setup, the interactive GraphDB condition scored higher than the static XML, JSON, and Turtle conditions. This comparison is limited by the two runs per condition and the incomplete 2x2 design, so it should not be interpreted as evidence that access mode generally supersedes data format. The interactive approach also revealed challenges in grounding, as the generation model frequently described its query strategy rather than explicitly tracing the instance-level identifiers responsible for its conclusions.

While the results are encouraging, certain limitations must be acknowledged. First, the answers were initially scored using a dedicated LLM-based grader. To mitigate potential grading errors, we performed targeted manual verification on outliers and complex edge cases, making minor adjustments where necessary. However, a complete manual review of all generated answers was not feasible, meaning the evaluation still relies heavily on an automated metric. Second, the evaluation was limited to two runs per condition, meaning the observed performance gaps between serialization formats lack statistical significance. Finally, the semantic queries were tested primarily against the standardized ArchiSurance model.

Future work should address these limitations by scaling the experimental runs to establish robust statistical baselines and validating the pipeline against large-scale, real-world enterprise repositories. Technical extensions could also incorporate structural graph analytics natively within the graph to further aid architectural decision-making.

References

- [1] Philipp-Lorenz Glaser et al. “Model-Based Construction of Enterprise Architecture Knowledge Graphs.” In: *Enterprise Design, Operations and Computing, 26th International Conference, EDOC 2022*. Springer Nature, 2022.
- [2] Philipp-Lorenz Glaser et al. “Exploring Enterprise Architecture Knowledge Graphs in Archi: The EAKG Toolkit.” In: *International Conference on Enterprise Design, Operations, and Computing*. Springer International Publishing, 2022.
- [3] The Open Group. *ArchiMate® 3.2 Specification*. Van Haren, 2023. ISBN: 978-94-018-0957-3.
- [4] The Open Group. *ArchiMate® Model Exchange File Format for the ArchiMate Modeling Language, Version 3.1*. Standard C19C. The Open Group, 2019.
- [5] RDFLib Team. *RDFLib: A Python library for working with RDF*. Open-source software project hosted on GitHub. 2024.
- [6] Ontotext AD. *GraphDB Documentation*. Ontotext. 2024.
- [7] The Open Group. *The TOGAF® Standard, 10th Edition*. Tech. rep. C220. The Open Group, 2022.
- [8] Henk Jonkers, Iver Band, and Dick Quartel. *ArchiSurance Case Study Version 2*. Case Study. The Open Group, 2016.
- [9] Nuno Miguel Silva, Miguel Mira da Silva, and Pedro Manuel Moreira Vaz Antunes de Sousa. “Modelling the Evolution of Enterprise Architectures Using Ontologies.” In: *2017 IEEE 19th Conference on Business Informatics (CBI)*. IEEE, 2017, pp. 79–88.
- [10] Josua Salentin and Simon Hacks. “Towards a Catalog of Enterprise Architecture Smells.” In: *15th International Conference on Wirtschaftsinformatik*. Potsdam, Germany, 2020.